INFORME 1

Hecho por Manuel Muñoz y Samuel Martínez

1. Introducción

El problema del productor-consumidor describe a dos procesos que comparten un búfer de tamaño fijo: el productor, encargado de generar datos y colocarlos en el búfer, y el consumidor, que los extrae para procesarlos. El desafío principal radica en la sincronización, ya que se debe evitar que el productor añada elementos si el búfer está lleno (desbordamiento) y que el consumidor intente quitarlos si está vacío (subdesbordamiento), garantizando además que ambos no modifiquen la estructura de datos simultáneamente para prevenir carreras críticas.
Las carreras críticas, por su parte, son el acceso simultáneo a regiones críticas por dos procesos o hilos distintos ejecutándose de forma paralela. Una región crítica es una parte del código donde un proceso accede a recursos compartidos y que deben ejecutarse de manera exclusiva para evitar errores.
En este programa se usan hilos para simular los procesos de productor y consumidor, creando un buffer de enteros con funcionamiento similar al de una estructura FIFO (First In, First Out) que leerá de un archivo una serie de números mientras los va sumando. Con esto, se observarán las carreras críticas que genera el problema.

2. Ejecución

Para la ejecución de este programa contamos con un Makefile, por lo que será necesario introducir los siguientes comandos por terminal desde la carpeta “ej1”:

```
samueelmtz@PortatilSamuel:~/Escritorio/SegundoCuatri/SOII/Practica3/practica3_soii/ej1$ make
gcc -Wall -Wextra -o bin/prod_cons src/prod_cons.o -lpthread
samueelmtz@PortatilSamuel:~/Escritorio/SegundoCuatri/SOII/Practica3/practica3_soii/ej1$ make run
```

El Makefile incluye -Wall y -Wextra para detectar posibles errores en la compilación, -lpthread para el uso de hilos y una opción de clean para borrar los ejecutables.

3. Funcionamiento

Este código implementa una solución clásica al problema del productor-consumidor utilizando hilos (*pthreads*) y un buffer circular compartido, aunque está diseñado deliberadamente para fallar debido a una condición de carrera. El hilo productor lee números de un archivo y los inserta en una cola FIFO, mientras que el hilo consumidor los extrae para sumarlos; sin embargo, ambos hilos utilizan una espera activa (bucles while que comprueban si el buffer está lleno o vacío) y carecen de mecanismos de sincronización como semáforos o mutex para proteger la región crítica. Para demostrar la vulnerabilidad, se incluyó funciones de retardo (sleep(1)) en medio de las operaciones de inserción y extracción, lo que provoca que los hilos se interrumpan entre sí y corrompan los índices y el

tamaño del buffer, resultando en una pérdida de integridad de los datos y sumas finales incorrectas.

4. Resultados

A continuación se muestra un fragmento de la ejecución:

```
Producido: 13 | Suma acumulada: 13
[CONS] Buffer vacio. Esperando...
[PROD] Insertado '13'. Inicio: 0 | Fin: 1 | Longitud: 1
Producido: 28 | Suma acumulada: 41
[CONS] Eliminado '13'. Inicio: 1 | Fin: 2 | Longitud: 0
Consumido: 13 | Suma acumulada: 13
[PROD] Insertado '28'. Inicio: 1 | Fin: 2 | Longitud: 2
Producido: 94 | Suma acumulada: 135
[PROD] Insertado '94'. Inicio: 2 | Fin: 3 | Longitud: 3
Producido: 91 | Suma acumulada: 226
[CONS] Eliminado '28'. Inicio: 2 | Fin: 4 | Longitud: 1
Consumido: 28 | Suma acumulada: 41
[PROD] Insertado '91'. Inicio: 3 | Fin: 4 | Longitud: 4
Producido: 96 | Suma acumulada: 322
[CONS] Eliminado '94'. Inicio: 3 | Fin: 5 | Longitud: 0
Consumido: 94 | Suma acumulada: 135
[CONS] Buffer vacio. Esperando...
[PROD] Insertado '96'. Inicio: 3 | Fin: 5 | Longitud: 5
Producido: 95 | Suma acumulada: 417
[PROD] Insertado '95'. Inicio: 4 | Fin: 6 | Longitud: 6
Producido: 66 | Suma acumulada: 483
[CONS] Eliminado '91'. Inicio: 4 | Fin: 7 | Longitud: 4
Consumido: 91 | Suma acumulada: 226
[PROD] Insertado '66'. Inicio: 5 | Fin: 7 | Longitud: 7
Producido: 27 | Suma acumulada: 510
[CONS] Eliminado '96'. Inicio: 5 | Fin: 8 | Longitud: 3
Consumido: 96 | Suma acumulada: 322
[PROD] Insertado '27'. Inicio: 6 | Fin: 8 | Longitud: 8
Producido: 53 | Suma acumulada: 563
[CONS] Eliminado '95'. Inicio: 6 | Fin: 9 | Longitud: 2
Consumido: 95 | Suma acumulada: 417
[PROD] Insertado '53'. Inicio: 7 | Fin: 9 | Longitud: 9
```

Las carreras críticas ocurren porque los hilos productor y consumidor acceden y modifican simultáneamente las variables globales de control (comp.tam, comp.inicio y comp.fin) sin ningún mecanismo de protección. En la captura, el desastre es evidente cuando la Longitud salta de forma incoherente (por ejemplo, de 0 a 5 o de 3 a 1), lo que demuestra que un hilo leyó un valor, se fue a dormir con el sleep(1), y cuando despertó para actualizarlo, el otro hilo ya había alterado la realidad del buffer, sobrescribiendo el dato anterior. Esto causa una corrupción de memoria donde el productor inserta elementos en índices equivocados y el consumidor resta unidades al tamaño basándose en valores obsoletos, invalidando por completo la integridad de la estructura FIFO y las sumas finales.
Como resultado de esto obtenemos:

```
[PROD] Suma acumulada: 4950
[CONS] Suma acumulada: 4298
```

La suma del productor debería ser equivalente a la del consumidor, por lo cual necesitaremos el uso de herramientas como mutexes para arreglar los errores producidos por las carreras críticas.

5. Conclusión

El problema del productor-consumidor por sí solo es un perfecto ejemplo de cómo se generan las conocidas como carreras críticas. Para la resolución de este problema, será conveniente el uso de variables de condición y mutexes (que se verá en el siguiente ejercicio).